> This PDF contains the final cram sheet. The full study book contains the linked topic guides. Website references need internet access.

Terraform Associate 004: Final Cram Sheet

Target: Terraform 1.12. **Reviewed:** 2026-10-14. Use this for final recall. Open the study map for examples.

1. Remember the model

Configuration: desired result. **Provider:** API plugin. **State:** recorded object bindings. **Plan:** proposed actions. **Apply:** execute actions and save state. IaC supports repeatability and review. Multi-cloud workflows still need provider-specific resource types.

2. Select the command

Need / Command

Install dependencies and initialize storage

```
terraform init
```

Select newer compatible dependencies

```
terraform init -upgrade
```

Check formatting without edits

```
terraform fmt -check
```

Check configuration consistency

```
terraform validate
```

Save proposed changes

```
terraform plan -out=tfplan
```

Apply that exact plan

```
terraform apply tfplan
```

Destroy managed resources

`terraform destroy`

`fmt` needs no initialization. `validate` needs installed dependencies. Validation does not prove cloud permissions. Plain `apply` replans. Saved-plan apply does not ask for approval again.

```
+ create    ~ update    - destroy
-/+ destroy then create    +/- create
then destroy
```

3. Separate version controls

Setting / Meaning

`required_version`

CLI constraint

`required_providers`

Provider sources and constraints

`provider` block

Connection settings

`.terraform.lock.hcl`

Selected provider versions and checksums

Module `version`

Registry module constraint

`~> 5.20` allows later 5.x versions. `~> 5.20.0` allows only later 5.20.x versions. The lock file covers providers, not modules or state writes. Commit it. Aliases select provider configurations, not different versions.

4. Predict the value and address

Local CLI precedence, weakest first:

```
default < TF_VAR_* < terraform.tfvars
< terraform.tfvars.json
```

```
< *.auto.tfvars /
*.auto.tfvars.json < -var / -var-file
```

Auto files load alphabetically. CLI assignments load in supplied order. Last wins. `prod.tfvars` needs `-var-file`. HCP has extra precedence rules.

Construct / Remember

`resource` / `data`

Manage an object / read information

List / set

Ordered with duplicates / unique members without numeric indices

Map / object

One element type / named attributes with specified types

`null` / unknown

No value / value not yet known

`count`

Numeric addresses: `thing.example[0]`

`for_each`

Keyed addresses: `thing.example["api"]`

`for` expression

Transform values

`dynamic`

Generate nested blocks

Resource `for_each` needs a map or set of strings. Keys must be known and non-sensitive. `merge(a, b)` gives conflicting keys to `b`. `try` handles dynamic errors. `can` tests successful evaluation.

5. Dependencies and conditions

References set dependencies. File order does not. Use `depends_on` for hidden dependencies.

Setting / Remember

> **`create_before_destroy`**
>
> Create replacement first. Naming and capacity limits still apply.

> **`prevent_destroy`**
>
> Reject destruction while the rule remains in configuration

> **`ignore_changes`**
>
> Ignore selected attributes during update planning

> **`validation`**
>
> Reject invalid inputs

`precondition`

Reject an invalid assumption

`postcondition`

Reject an invalid result. Block dependent work.

`check`

Warn on a false assertion

Unknown values can defer conditions. Outputs allow preconditions, not postconditions.

6. Modules and state operations

Pass child inputs explicitly. Read `module.name.output`. Call child directories with `module` blocks. Map provider aliases explicitly. Use `version` for registry modules and `?ref=` for Git sources.

Need / Use

Inspect addresses

`terraform state list`

Inspect one object

`terraform state show ADDRESS`

Rename without replacing

`moved` block

Stop management, retain object

`removed` with `destroy = false`

Adopt an existing object

`import` block or `terraform import`

Update state without changing infrastructure

Refresh-only plan/apply

Copy state to another backend

`terraform init -migrate-state`

CLI import does not generate configuration. Refresh-only does not edit it. Normal apply can reverse drift. Remote state does not imply remote execution. Force-unlock only your confirmed stale lock. Locking support depends on the backend.

7. Secrets and logs

Sensitive hides. Ephemeral omits.

- Sensitive values can remain in state and plans.
- Ephemeral values need permitted contexts. Root outputs cannot be ephemeral.
- Write-only arguments are not retained.
- Normal Vault data sources can store secrets in state.
- `TF_LOG=TRACE`: maximum detail. `TF_LOG_PATH`: log file. A path alone enables nothing.

8. HCP Terraform

Term / Remember

Workspace

Configuration context, state, variables, and runs

Project

Group workspaces and access

CLI workspace

Separate state for one working configuration

CLI/VCS/API workflow

How a run starts

Remote/agent/local execution

Where Terraform runs

`cloud` block

HCP integration. Cannot coexist with `backend`.

`terraform login`

HCP authentication, not cloud-provider authentication

Variable set

Shared inputs. Priority sets can override narrower scopes.

Run trigger

Queue a downstream run after a successful upstream apply

Sentinel / OPA

Policy enforcement

Health assessment

Detect drift and failed checks

A trigger does not pass outputs. A speculative plan cannot be applied. Also recognize teams, private registry, run tasks, Explorer, change requests, and dynamic credentials.

Coverage follows the 004 objectives. Use the topic guides for references and self-tests.